BRIEFINGIQ · ORACLE CLOUD INFRASTRUCTURE

BriefingIQ AI Assistant

A natural-language, **tool-calling AI agent** embedded in the BriefingIQ platform — it plans, queries, books, drafts and writes back, with **server-side role-based access control** on every byte it reads. The entire stack — app, data, search and LLM — runs inside one OCI Virtual Cloud Network.

OCI · VCN-contained

Oracle GenAI · reasoning

RBAC · enforced server-side

~25 tools · read & write

02 EXECUTIVE SUMMARY

What we've built

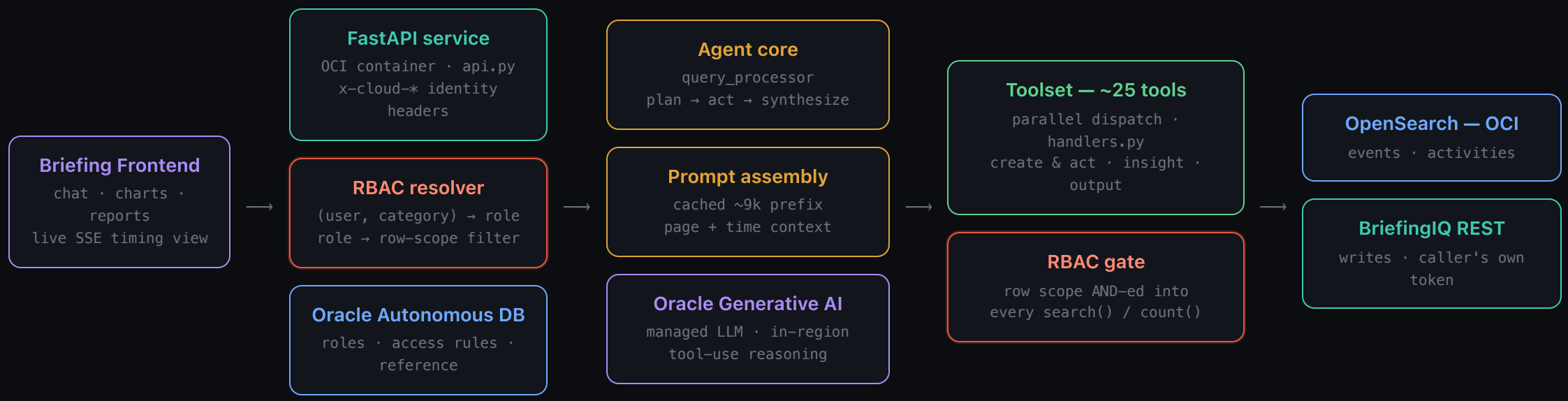
Users talk to BriefingIQ in plain English. The agent doesn't just answer — it **acts**: it searches history, checks rooms, drafts agendas and emails, and writes complete briefings back into the platform.

- **A real agent loop, not a chatbot** — the model plans, calls tools, reads results, and keeps going until the task is done.
- **Full read path** — event & activity history, counts, reports, charts, PDFs.
- **Full write path** — creates complete briefings, edits briefing fields, pushes agendas, books rooms — always through the caller's own credentials.
- **Streaming UX** — the UI shows every LLM and tool step live over SSE, with per-query token and cost accounting.
- **RBAC by construction** — the caller's role is resolved from the Oracle access model and enforced at a single server-side gate the model cannot bypass.
- **Engineered for latency & cost** — prompt caching, parallel tool fan-out, server-side date resolution.
- **Provider-agnostic core** — the reasoning layer is an adapter; the agent loop, tools and RBAC don't change if the model does.

03 ARCHITECTURE

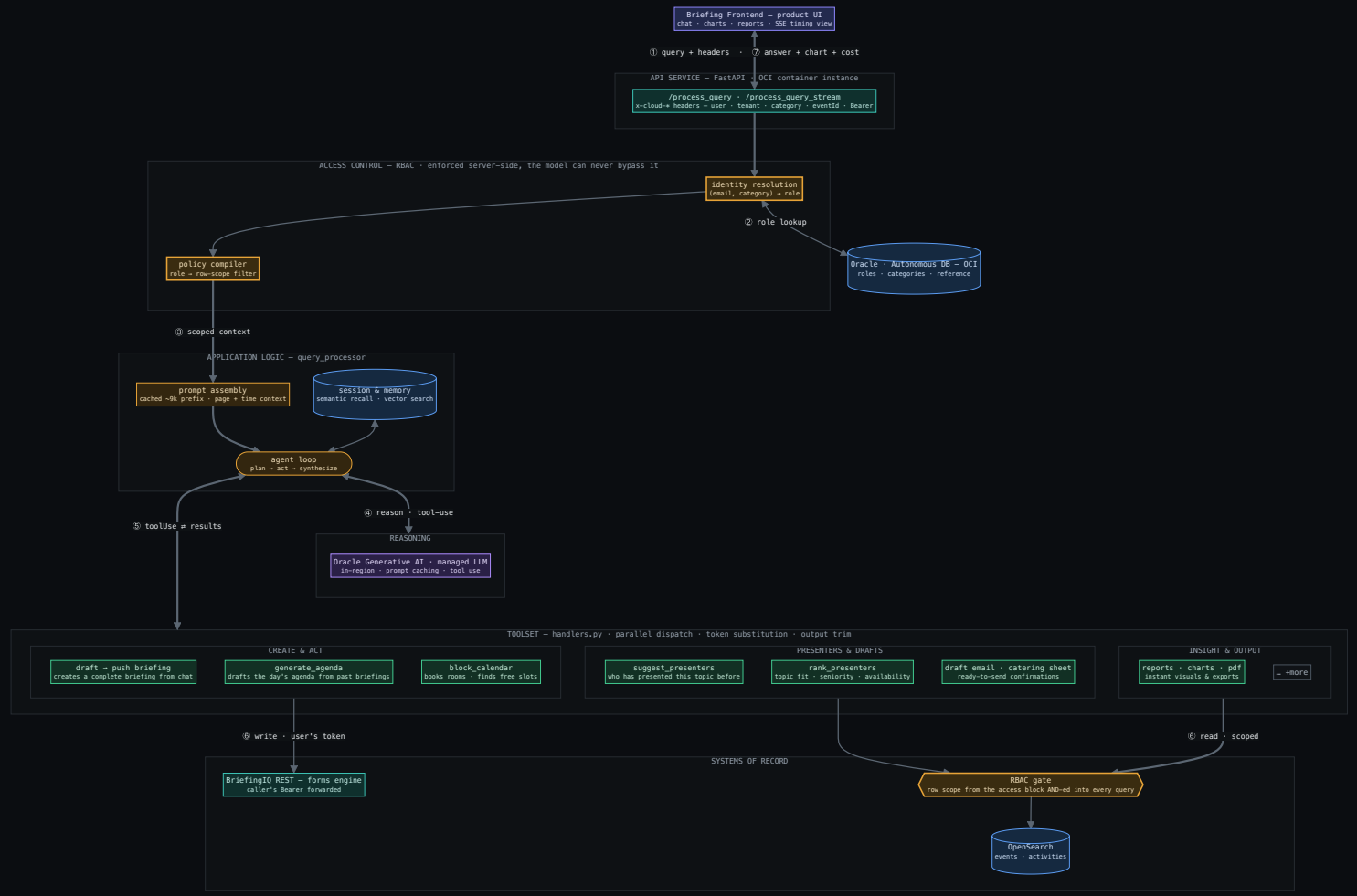
The system at a glance

Identity and row-level scope are resolved **before the agent runs** and enforced at the gate in front of the data. The model reasons and requests tools — it never controls what it may see.



reads → RBAC gate · writes → user's own token · reasoning → Oracle GenAI

Every edge is a real call — follow ① → ⑦



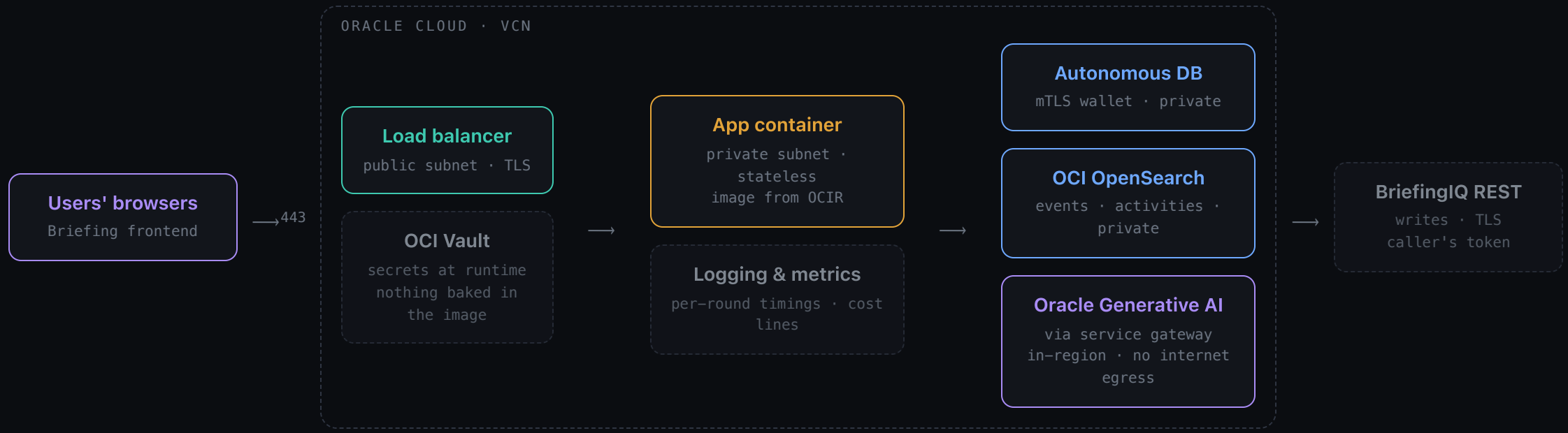
bold edges — numbered request path · identity resolved before the agent runs · reads pass the RBAC gate

logical view · detail

05 DEPLOYMENT

One VCN holds everything

App, data, search, secrets and the LLM are all **Oracle Cloud services**. The app sits in a private subnet behind a TLS load balancer; Oracle GenAI is reached over the Oracle service network — **traffic never crosses the public internet**.



OCIR images · Vault secrets · IAM dynamic-group policies — no credentials on disk

deployment view

Data protection — what the platform docs say

This is what the linked provider documentation states, for **OCI Generative AI** and **AWS Bedrock** alike.

PRIVATE NETWORK PATH

Model traffic can stay off the public internet

OCI Generative AI supports **private endpoints** — a private IP inside your VCN, reachable only per your routing rules, security lists and NSGs. AWS Bedrock offers the equivalent via **VPC interface endpoints (PrivateLink)**, with traffic monitorable through VPC flow logs.

MODEL PRIVACY

Model providers can't see your content

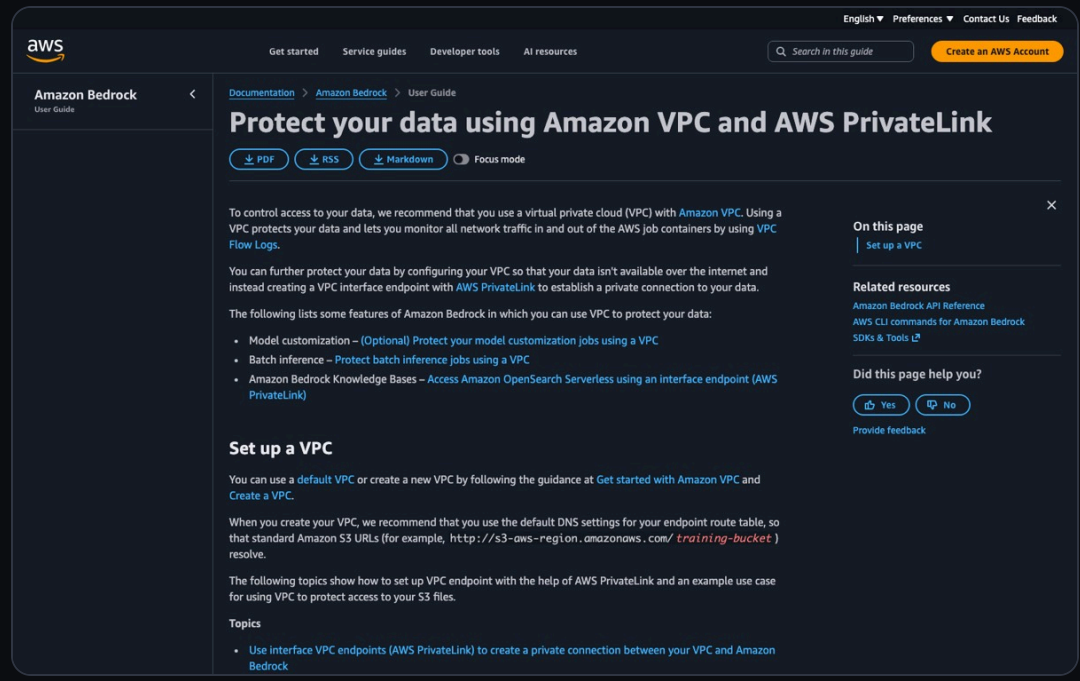
Bedrock runs each provider's model in **service-owned deployment accounts** the provider cannot access — so providers never see prompts or completions. OCI GenAI serves models from Oracle's service tenancy over the private path above.

ENCRYPTION & AUDIT

Encrypted in transit and at rest

Both platforms require **TLS 1.2+** in transit and encrypt stored data, with managed keys (OCI Vault / AWS KMS) and audit logging (OCI Audit / CloudTrail).

OCI · generative-ai/private-endpoint · generative-ai/private-network-access
AWS · bedrock/data-protection · bedrock/usingVPC



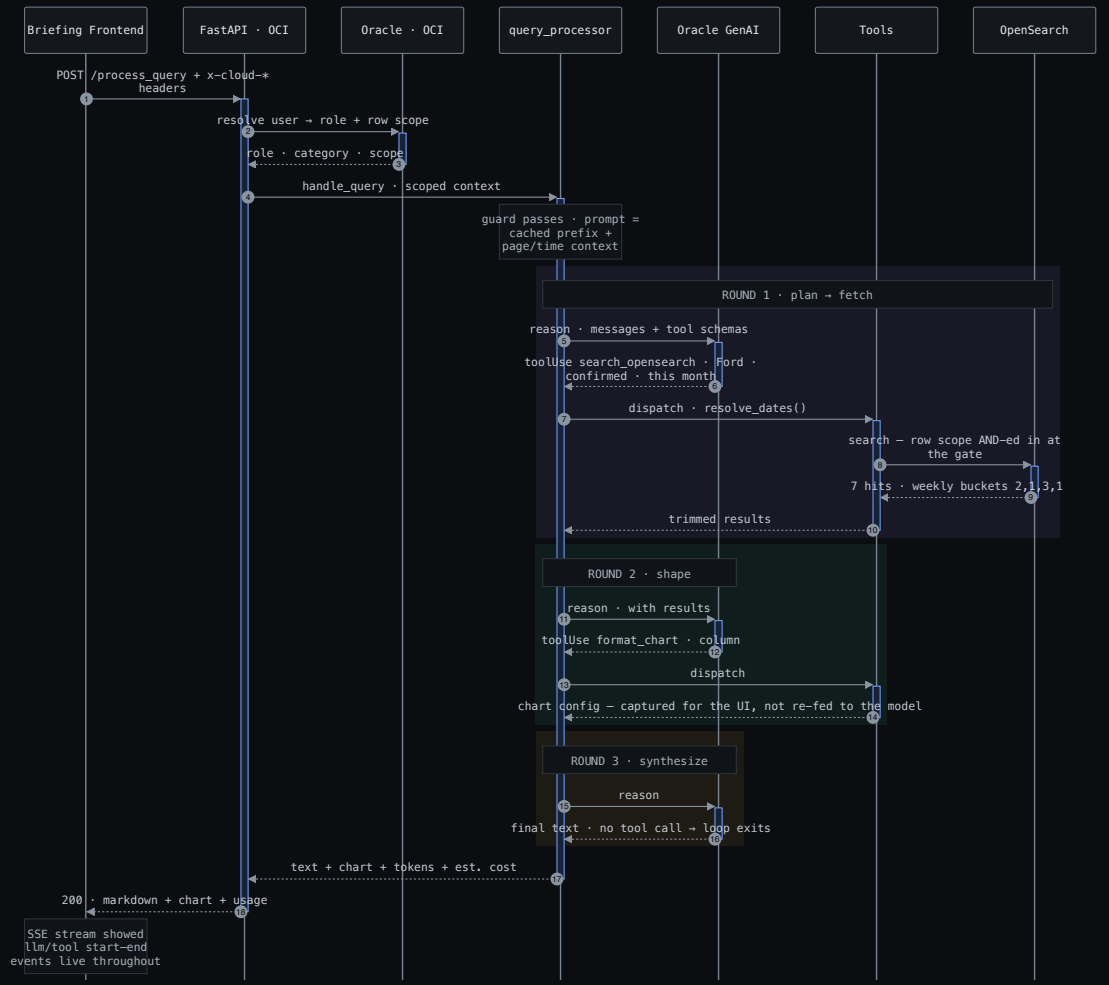
07 REQUEST WALKTHROUGH

Life of a request — end to end

Example: *"How many confirmed briefings does Ford have this month? Show it as a chart."*

①	**Query arrives with identity**	Frontend POSTs the prompt + `x-cloud-*` headers — user, tenant, category, page context, Bearer token.
②	**Role & scope resolved from the DB**	The service looks up the caller's roles and row-access rules in the access-model tables — before any reasoning happens.
③	**Prompt assembled**	Cached ~9k-token system prefix + page and time context. Pasted query syntax is rejected before the model sees it.
④	**Round 1 · plan → fetch**	Model requests `search_opensearch`; the server resolves relative dates and **ANDs the RBAC filter in at the gate**. 7 hits return.
⑤	**Round 2 · shape**	Model calls `format_chart` — the chart config is captured for the UI, not re-fed to the model.
⑥	**Round 3 · synthesize**	Model writes the answer, no further tool call — the loop exits.
⑦	**Answer + chart + cost**	Markdown, chart config, token counts and estimated cost return; the SSE stream showed every step live throughout.

The same request — every call, end to end



every arrow is a real call or return · shaded blocks are the agent loop's three rounds — plan → fetch · shape · synthesize

09 REASONING ENGINE

The agent loop

One loop, provider-agnostic: the model reasons through **Oracle Generative AI**, requests tools, the server executes them and feeds results back — until the answer is ready.

HOW IT THINKS

- **Plan → act → synthesize** — the model decides which tools it needs and in what order.
- **Parallel fan-out** — concurrent tool calls run on a thread pool; total latency is the *max*, not the sum.
- **Session & memory** — recent turns replayed each round, plus persistent per-user memory with semantic (vector) recall of similar past briefings.
- **Guardrails in the loop** — index allowlist, banned query keys, output trimming, step budget.

WHY IT'S FAST & AFFORDABLE

- **Prompt caching** — the ~9k-token static prefix (instructions, tool schemas, field reference) is cache-read on every round, only the delta is re-billed.
- **Cost transparency** — every response carries input/output token counts and an estimated cost line.
- **Structured logs** — every round, tool call and cost is a log line; latency is measurable per stage.

BriefingIQ

The toolset — what the agent can do

Everything the agent does happens through a **tool** — a typed function the server executes. The model can only do what a tool allows, and every tool inherits the caller's identity, tenant and RBAC context from the request. Six groups, ~25 tools.

Briefing — create & edit

Draft a **complete briefing** from chat and push it; edit any field of an existing one through the platform API.

Agenda & presenters

Generate EBC-style agendas from past briefings; rank presenters by topic fit and track record; push the agenda in.

Scheduling

Rooms, existing bookings, free-slot computation, conflict-checked calendar blocking.

Insight & search

Scoped OpenSearch queries and counts over events & activities — history, status, attendees, topics.

Drafts

Ready-to-send email and catering-sheet drafts, grounded in the actual event data.

Output

Charts (9 types), report tables with typed columns, downloadable PDFs.

Tools that create, book and edit

Every mutation goes through the BriefingIQ REST API with the **caller's own Bearer token** forwarded — so the platform's own permissions govern every write, and every change is attributable to the user.

`briefing_builder — draft → push briefing`
Builds a **complete briefing from a chat conversation**: creates the event, then creates and fills the VISIT_INFO record — objective, company detail, host, and the controlled-vocabulary lookup fields — exactly as the platform's forms engine expects.

`briefing_editor — edit any field`
Updates fields on an existing briefing through the caller's API session, so **server-side platform RBAC applies to the edit itself**. Field options come from the platform's own controlled vocabularies.

`generate_agenda`
Drafts the day's agenda from similar past briefings + the presenter engine, then an LLM writes the sessions.

`push_agenda_to_briefingiq`
Turns approved sessions into real calendar activities — rooms first (`get_event_rooms`), then **only after the user confirms**.

`block_calendar`
Reserves a room window with a **pre-flight conflict check** — overlapping bookings return a conflict instead of a double-book.

`find_vacant_slots · get_resource_schedule · list_rooms`
Free-window computation respecting working hours; live schedules per room; bookable-room discovery per tenant.

Tools that search, report and draft

`search_opensearch · count_opensearch`
Model-authored DSL over **events** (attendees, status, location, opportunity) and **activities** (topics, presenters, rooms, catering) — every query intersected with the caller's RBAC filter at the gate.

`suggest_presenters`
Who has presented this topic, to this industry, and been accepted? Ranked by accepted sessions and recency, with graceful fallbacks.

`format_chart`
Nine chart types (column, line, pie, heatmap, xrange…) rendered natively by the frontend — the config goes straight to the UI, not back through the model.

`drafts — email & catering`
Ready-to-send confirmation emails and catering sheets, grounded in the actual event data.

`generate_report`
Typed report tables — event, customer, status, location, presenter, topic columns — built straight from a scoped query.

`generate_pdf · get_time_context`
Downloadable PDF exports; server-side time context (day boundaries, epoch-ms) for reliable date filtering.

Where the data lives

Three backends, each with one clear job.

OPENSEARCH · OCI

Briefing history — what the agent searches

Every briefing and every session, indexed and searchable: customer, status, attendees, location, topics, presenters, rooms, catering. All questions, counts, reports and agenda suggestions come from here.

- **Index allowlist** — the agent can touch `events` and `activities`, nothing else in the cluster.
- **Every read scoped** — the RBAC filter is AND-ed in at the client chokepoint.

ORACLE AUTONOMOUS DB · OCI

Access rules — who may see what

The same Oracle database the platform itself uses stores the roles and access rules. The agent reads them to decide what each user is allowed to see — one source of truth, nothing duplicated.

- **Role resolution** — (user, category) → active roles.
- **Rule tables** — per-role row rules, cached with a short TTL.

BRIEFINGIQ REST

Where the agent writes changes

When the agent creates a briefing, pushes an agenda or books a room, it calls BriefingIQ's own API with the signed-in user's token — the change lands exactly as if the user made it by hand.

- **No service account** — the agent holds no write credential of its own.
- **The platform checks every write** — its own permissions apply, per user, per request.

RBAC — the access model

The agent reuses the **platform's own Oracle access model** — no parallel permission system to drift out of sync. Identity comes from the request; rules come from the database.

1	**Identity from the request**	`x-cloud-*` headers carry the signed-in user and category — set by the platform, not the model.
2	**Roles from Oracle**	The user's active roles are resolved from the event-role tables (12 roles — requester, host, admin, coordinator…).
3	**Rules per role**	Each role maps to row rules — `(property, operation, value)` — from `M_EVENT_ROLE_DATA_ACCESS_MAP`, cached 5 min.
4	**Compiled to a filter**	Rules are translated into OpenSearch `bool.filter` clauses — Oracle property names mapped to index field paths.

COMBINATION POLICY

- **Within a role** — rules combine by the rule's own condition (AND / OR), default AND.
- **Across a user's roles** — OR: any role that grants access wins (most-permissive union).
- **"My own" rows** — `$loggedInUser` ownership expands to *host of the event OR requester of it*.
- **Unknown or rule-less role** — falls back to the **most-restrictive** shape: own/hosted requests only.

Enforcement the model can't bypass

The filter isn't a prompt instruction — it's **code, at a single chokepoint**, applied after the model's query is parsed and before it reaches the data.

```
# every read funnels through one gate
def search(index, dsl_query):
    q = normalize(dsl_query)          # model's query
    scope = access_context()          # resolved at request start
    q = AND(q, scope.filter)          # cannot be widened
    return opensearch.search(index, q)

# resolution failed? → fail closed
if not scope.resolved: deny()       # never run unfiltered
```

`events` role filter wrapped around every query

`activities` scoped via the allowed-event join · pinned-event checks · overflow ⇒ deny

`any other index` denied outright

WHY THIS HOLDS

- **Single chokepoint** — one `search()/count()` path; there is no unfiltered route to the data.
- **Server-side, post-parse** — the scope is AND-ed after the model's DSL is normalized; a crafted query cannot widen past it.
- **Fail closed** — if role resolution errors, the context is marked unresolved and reads are denied, never run open.
- **Defense in depth** — index allowlist + banned query keys + pasted-DSL rejection before the model, platform token on every write after it.
- **Resolved once per request** — scope is fixed before reasoning starts; nothing the model says changes it.

BRIEFINGIQ · AI ASSISTANT

An agent that acts, inside walls that hold.

Every read scoped by the Oracle access model. Every write on the user's own credentials. Every service — app, data, search, secrets, LLM — **inside one VCN**.

Live · agent + tools + RBAC gate

Live · session & per-user memory

Live · one VCN, end to end

Thank you